

1. Translate the following Java statements into statements in our programming language.

- a. $x = (y == 2) ? 5 + x : 6;$
 $x := \text{if } y = 2 \text{ then } 5 + x \text{ else } 6 \text{ fi}$
- b. $\text{if } (y == 2) \{x = 5 + x;\} \text{ else } \{x = 6;\}$
 $\text{if } y = 2 \text{ then } x := 5 + x \text{ else } x := 6 \text{ fi}$
- c. $\text{if } (y == 2) \{x = 5 + x;\} x = 6;$
 $\text{if } y = 2 \text{ then } x := 5 + x \text{ fi}; x := 6$ or $\text{if } y = 2 \text{ then } x := 5 + x \text{ else skip fi}; x := 6$

2. Create a program that calculates the power of 2. We run it with input integer n and returns $y = 2^n$; unless $n < 0$, in which case we return 0.

If we write it with indentation, then one way to write it is as follows.

```

if  $n < 0$  then
     $y := 0$ 
else
     $x := 0;$ 
     $y := 1;$ 
    while  $x < n$ 
    do
         $x := x + 1;$ 
         $y := y + y$ 
    od
fi

```

It is also acceptable to write it in one line:

```

if  $n < 0$  then  $y := 0$  else  $x := 0; y := 1; \text{ while } x < n \text{ do } x := x + 1; y := y + y \text{ od fi}$ 

```

3. Translate the following C statements into statements in our programming language.

- a. $x = a * ++z$
 $z := z + 1; x := a * z$
- b. $x = a * z++$
 $x := a * z; z := z + 1$
- c. **while** $(--x >= 0)$ $z *= x;$
 $x := x - 1; \text{ while } x \geq 0 \text{ do } z := z * x; x := x - 1 \text{ od}$
- d. **while** $(x-- >= 0)$ $z *= x;$
while $x \geq 0 \text{ do } x := x - 1; z := z * x \text{ od}; x := x - 1$

Operational Semantics of Statements

To analyze how a program works, we'll start with operational semantics: We'll model execution as a sequence of "configurations" — snapshots of the program and memory state — over time. The semantics rules describe how step-by-step execution of the program changes memory. When the program is complete, we have the final memory state of execution.

- A **configuration** $\langle S, \sigma \rangle$ is an ordered pair of a statement and state.
 - The **operational semantics** of programs is given by a relation on configurations: $\langle S, \sigma \rangle \rightarrow \langle S_1, \sigma_1 \rangle$ means that executing S in state σ for *one step* yields $\langle S_1, \sigma_1 \rangle$. S_1 is called the **continuation** of S (or $\langle S, \sigma \rangle$). The " $\rightarrow$ " here means "evaluate a configuration for one step".
 - For example, $\langle x := x + 1; y := x, \{x = 5\} \rangle \rightarrow \langle y := x, \{x = 6\} \rangle$. In this example, $y := x$ is the continuation of $x := x + 1; y := x$.
4. Evaluate configuration $\langle x := x + 1; y := x, \{x = 5\} \rangle$ to completion.
 $\langle x := x + 1; y := x, \{x = 5\} \rangle \rightarrow \langle y := x, \{x = 6\} \rangle \rightarrow \langle E, \{x = 6, y = 6\} \rangle$
- If there is a sequence of configurations $\langle S, \sigma \rangle \rightarrow \dots \rightarrow \langle E, \tau \rangle$, then we say **execution of S starting in state σ converges to (or terminates in) state τ** . Based on which part we want to emphasize, we can simplify this sentence in different ways: " S converges", " σ terminates in state τ " or " σ terminates".
 - The opposite of convergence is **divergence**, which means the program doesn't finish. For us, that means infinite loops, or in other words, an infinitely long sequence of calculations. For example, $\langle \text{while } x = 0 \text{ do } x := 0 \text{ od}, \{x = 0\} \rangle$ and $\langle \text{while } x \geq 0 \text{ do } x := x + 1 \text{ od}, \{x = 0\} \rangle$ diverge.

(Operational Semantics Rules)

- For a no-op statement: $\langle \text{skip}, \sigma \rangle \rightarrow \langle E, \sigma \rangle$
 - For an assignment statement:
 - $\langle v := e, \sigma \rangle \rightarrow \langle E, \sigma[v \mapsto \sigma(e)] \rangle$
 - $\langle b[e_1] := e, \sigma \rangle \rightarrow \langle E, \sigma[b[e_1] \mapsto \sigma(e)] \rangle$
5. Evaluate the following configurations.
- a. $\langle x := 5 * x + 6, \{x = 1, y = 2\} \rangle \rightarrow \langle E, \{x = 11, y = 2\} \rangle$
- b. With $\sigma(x) = 8$, $\langle b[x + 1] := x * 5, \sigma \rangle \rightarrow \langle E, \sigma[b[9] \mapsto 40] \rangle$
- For a conditional statement:
 - $\langle \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma \rangle \rightarrow \begin{cases} \langle S_1, \sigma \rangle, & \text{if } \sigma(B) = T \\ \langle S_2, \sigma \rangle, & \text{if } \sigma(B) = F \end{cases}$
 - $\langle \text{if } B \text{ then } S_1 \text{ fi}, \sigma \rangle \rightarrow \begin{cases} \langle S_1, \sigma \rangle, & \text{if } \sigma(B) = T \\ \langle \text{skip}, \sigma \rangle, & \text{if } \sigma(B) = F \end{cases}$
6. Let $S \equiv \text{if } x > 0 \text{ then } y := 0 \text{ fi}$,
- a. Evaluate S in state σ , where $\sigma(x) = 5$.
 $\langle \text{if } x > 0 \text{ then } y := 0 \text{ fi}, \sigma \rangle \rightarrow \langle y := 0, \sigma \rangle \rightarrow \langle E, \sigma[y \mapsto 0] \rangle$
- b. Evaluate S in state σ' , where $\sigma'(x) < -2$.

$\langle \text{if } x > 0 \text{ then } y := 0 \text{ fi}, \sigma' \rangle \rightarrow \langle \text{skip}, \sigma' \rangle \rightarrow \langle E, \sigma' \rangle$

- For an iterative statement:

- Let $W \equiv \text{while } B \text{ do } S \text{ od}$, then $\langle W, \sigma \rangle \rightarrow \begin{cases} \langle E, \sigma \rangle, & \text{if } \sigma(B) = F \\ \langle S; W, \sigma \rangle, & \text{if } \sigma(B) = T \end{cases}$

Here, the second case is the only operational semantic rule that produces a continuation that is larger than the starting statement. Thus, if the loop is infinite, we will get infinite configurations.

So far, all the rules we have seen are considered as axioms. For sequence statements, it is slightly more complicated to describe the rules, but it is also easy to understand.

- For a sequence statement:

- If $\langle S_1, \sigma \rangle \rightarrow \langle U, \sigma_1 \rangle$ then $\langle S_1; S_2, \sigma \rangle \rightarrow \langle U; S_2, \sigma_1 \rangle$
- If $\langle S_1, \sigma \rangle \rightarrow \langle E, \sigma_1 \rangle$ then $\langle S_1; S_2, \sigma \rangle \rightarrow \langle S_2, \sigma_1 \rangle$

7. Let $S \equiv s := 0; k := 0; W$, where $W \equiv \text{while } k < n \text{ do } S_1 \text{ od}$ and $S_1 \equiv s := s + k + 1; k := k + 1$, evaluate S in state σ with $\sigma(n) = 1$.

$$\begin{aligned} \langle S, \sigma \rangle &= \langle s := 0; k := 0; W, \sigma \rangle \\ &\rightarrow \langle k := 0; W, \sigma[s \mapsto 0] \rangle \\ &\rightarrow \langle W, \sigma[s \mapsto 0][k \mapsto 0] \rangle \\ &= \langle \text{while } k < n \text{ do } S_1 \text{ od}, \sigma[s \mapsto 0][k \mapsto 0] \rangle \\ &\rightarrow \langle S_1; W, \sigma[s \mapsto 0][k \mapsto 0] \rangle \quad // \text{ since } \sigma[s \mapsto 0][k \mapsto 0](k < n) = T \\ &= \langle s := s + k + 1; k := k + 1; W, \sigma[s \mapsto 0][k \mapsto 0] \rangle \\ &\rightarrow \langle k := k + 1; W, \sigma[s \mapsto 1][k \mapsto 0] \rangle \\ &\rightarrow \langle W, \sigma[s \mapsto 1][k \mapsto 1] \rangle \\ &= \langle \text{while } k < n \text{ do } S_1 \text{ od}, \sigma[s \mapsto 1][k \mapsto 1] \rangle \\ &\rightarrow \langle E, \sigma[s \mapsto 1][k \mapsto 1] \rangle \quad // \text{ since } \sigma[s \mapsto 1][k \mapsto 1](k < n) = F \end{aligned}$$

We have seen the step-by-step operational semantics for our program now, but we sometimes do not need that many operation details. We want to learn some ways to simplify this process to focus on the most important configurations.

- If we have a sequence of configurations $\langle S_0, \sigma_0 \rangle \rightarrow \langle S_1, \sigma_1 \rangle \rightarrow \langle S_2, \sigma_2 \rangle \rightarrow \dots \rightarrow \langle S_n, \sigma_n \rangle$, then we say $\langle S_0, \sigma_0 \rangle$ **evaluates to** $\langle S_n, \sigma_n \rangle$ **in n steps**, which is denoted as $\langle S_0, \sigma_0 \rangle \rightarrow^n \langle S_n, \sigma_n \rangle$.
 - “ n ” can even be 0 $\langle S_0, \sigma_0 \rangle \rightarrow^0 \langle S_0, \sigma_0 \rangle$.

Under many circumstances, we do not really care the value of n in the above definition, we can use “ $\rightarrow^*$ ” to represent **any number of steps**.

- Program S starting in σ terminates in (converges to) τ if $\langle S, \sigma \rangle \rightarrow^* \langle E, \tau \rangle$.

8. Let $S \equiv s := 0; k := 0; W$, where $W \equiv \text{while } k < n \text{ do } S_1 \text{ od}$ and $S_1 \equiv s := s + k + 1; k := k + 1$, evaluate S in state σ with $\sigma(n) = 3$.

$$\begin{aligned} \langle S, \sigma \rangle &\rightarrow^0 \langle s := 0; k := 0; W, \sigma \rangle \\ &\rightarrow^2 \langle W, \sigma[s \mapsto 0][k \mapsto 0] \rangle \\ &\rightarrow^* \langle W, \sigma[s \mapsto 1][k \mapsto 1] \rangle \quad // \text{after each iteration of } W \\ &\rightarrow^* \langle W, \sigma[s \mapsto 3][k \mapsto 2] \rangle \\ &\rightarrow^* \langle W, \sigma[s \mapsto 6][k \mapsto 3] \rangle \\ &\rightarrow \langle E, \sigma[s \mapsto 6][k \mapsto 3] \rangle \end{aligned}$$